

DevOps, CI/CD e Cultura de Entrega

Processos de Software – Sprint 2

Prof. Fernando · UFRN · 2026.1

Contexto: de onde viemos

Sprint 1	Sprint 2
Processo em operação	Processo automatizado
Quadro Kanban manual	Pipeline CI/CD
"Funciona na minha máquina"	Docker + reprodutibilidade
Retrospectiva qualitativa	Métricas DORA quantitativas

Sprint 1 foi sobre colocar o processo para rodar. Sprint 2 é sobre **automatizar e medir**.

O que DevOps NÃO é

✗ DevOps não é um cargo

Não existe "o DevOps da equipe". É cultura, não pessoa.

✗ DevOps não é uma ferramenta

Jenkins, GitHub Actions, Docker — são meios, não fins. Usar Jenkins não faz você ter DevOps, assim como ter quadro Kanban não faz você ser ágil.

✗ DevOps não é uma equipe separada

Criar um "time de DevOps" entre Dev e Ops é o oposto do que o movimento propõe.

✗ DevOps não é "infra automatizada"

Automação é parte, mas sem cultura, colaboração e medição, não funciona.

O que DevOps É

Conjunto de **práticas culturais e técnicas** que reduz o tempo entre uma ideia e ela estar rodando em produção, **com qualidade e confiança**.

Três ideias centrais:

1. **Colaboração** entre quem desenvolve e quem opera
2. **Automação** do que é repetitivo e propenso a erro
3. **Medição** contínua para melhorar com dados

O modelo mais usado para descrever essa cultura é o **CALMS**.

CALMS – os 5 pilares

Pilar	Significado
C — Culture	Responsabilidade compartilhada, colaboração, sem silos
A — Automation	Se faz manualmente mais de 2x, automatize
L — Lean	Fluxo contínuo, eliminar desperdício, reduzir WIP
M — Measurement	Medir para melhorar, não para punir
S — Sharing	Post-mortems sem culpa, documentação viva, transparência

— Jez Humble & Gene Kim

Notem o **L de Lean**: DevOps é Lean aplicado à entrega de software. Não é coincidência que vocês vejam Lean antes de DevOps.

C – Culture: responsabilidade compartilhada

No modelo tradicional:

Dev: "Eu escrevo o código, pronto, meu trabalho acabou"
Ops: "Vocês me deram algo que não funciona em produção"

Em DevOps:

Equipe: "Somos responsáveis coletivamente
desde o código até a produção"

Na prática do projeto: se o deploy quebra, não é "culpa de quem fez o merge". É um problema **do time** — e vai para a retrospectiva.

Isso vocês já viram em Scrum: o time é responsável coletivamente pelo incremento.

A – Automation: eliminar trabalho manual

Regra: se você faz manualmente mais de duas vezes, automatize.

Manual (Sprint 1)	Automatizado (Sprint 2)
<code>go test</code> no terminal	GitHub Actions roda a cada push
"Funcionou na minha máquina"	Docker garante mesmo ambiente
Verificar formatação com olho	<code>gofmt</code> + <code>go vet</code> no CI
Deploy copiando arquivos	Pipeline de deploy automatizado

Automação não é luxo. É o que **libera o time para pensar** em vez de repetir.

L – Lean: fluxo contínuo de entrega

DevOps = Lean aplicado ao pipeline de software:

Conceito Lean	Em DevOps
Fluxo contínuo	Pipeline CI/CD sem interrupções
Reduzir WIP	Branches curtas, merge frequente
Pull system	Deploy on demand (não por sprint)
Gestão visual	Dashboard do pipeline
Muda (desperdício)	Build manual, teste manual, espera por aprovação

A conexão é direta: **Kanban** → **pipeline CI/CD**. As colunas do quadro viram estágios do pipeline.

M – Measurement: medir para melhorar

Não é: "quantas linhas de código por dia"

É: "quanto tempo leva do commit ao deploy?"

Métricas que importam (**DORA**):

Métrica	Pergunta
Deployment Frequency	Com que frequência fazemos deploy?
Lead Time for Changes	Quanto tempo do commit ao deploy?
Change Failure Rate	Quantas mudanças falham em produção?
Time to Restore	Quanto tempo para restaurar serviço?
Rework Rate (2025)	Quanto retrabalho geramos?

Medir sem punir. A métrica serve para a equipe **aprender**, não para ranking.

S – Sharing: transparência e aprendizado

Blameless Post-Mortem: quando algo quebra, a pergunta não é *"quem fez isso?"* mas *"o que no nosso sistema permitiu que isso acontecesse?"*

Princípios:

- Documentação viva (não em PDF esquecido)
- Conhecimento compartilhado (ninguém é indispensável)
- Decisões registradas (ADRs — Architecture Decision Records)
- Erros publicados para todos aprenderem

A equipe que esconde erros repete erros. A equipe que compartilha erros evolui.

Os Três Caminhos

Gene Kim – *The DevOps Handbook*

Os princípios fundamentais que guiam todas as práticas DevOps.

Caminho	Foco	Direção
1º — Fluxo	Otimizar entrega	Dev → Produção (→)
2º — Feedback	Criar loops rápidos	Produção → Dev (←)
3º — Experimentação	Cultura de aprendizado	↻ Melhoria contínua

1º Caminho: Fluxo (→)

Otimizar o fluxo da **esquerda para a direita**: do desenvolvimento até a produção.

Práticas:

- Tornar o trabalho **visível** (Kanban → pipeline dashboard)
- Reduzir **batch sizes** (lotes menores = feedback mais rápido)
- **Limitar WIP** (você já fazem isso no Kanban!)
- Nunca **passar defeitos adiante** (se o teste falha, o código não avança)

Na prática: pipeline de CI/CD que roda testes automaticamente a cada commit. Se o teste falha, o código não avança. Simples, mas poderoso.

Regra de ouro: se não está automatizado no pipeline, não conta como feito.

2º Caminho: Feedback (←)

Criar loops de feedback rápidos da **direita para a esquerda**: produção informa desenvolvimento.

Loops de feedback (do mais rápido ao mais lento):

Loop	Tempo	Exemplo
Testes automatizados	Segundos	<code>go test</code> no CI
Code review	Horas	Pull request
Testes de integração	Minutos	Pipeline completo
Monitoramento	Contínuo	Alertas de produção
Retrospectiva	Semanas	Sprint review

Quanto **mais rápido** o feedback, **menor o custo** de corrigir.

3º Caminho: Experimentação (v)

Cultura de **experimentação e aprendizado contínuo**.

- Correr **riscos calculados**
- Aprender com **falhas** (blameless post-mortems)
- **Kaizen** — melhoria contínua a cada iteração
- Deploy frequente, mesmo de coisas pequenas
- **Feature flags** para testar em produção com segurança

A equipe que tem **medo de fazer deploy** está presa no oposto de DevOps. A equipe que faz deploy **10 vezes por dia** confia no pipeline.

CI – Integração Contínua

A prática de integrar código frequentemente — **no mínimo diariamente** — e validar cada integração com build e testes automatizados.

Não é "ter Jenkins". É o ato de fazer integração na branch de produção várias vezes ao dia, com confiança de que os testes passam.

Se o time faz merge **uma vez por sprint**, não tem CI.

CD – Entrega / Deploy Contínuo

Termo	Significado
Continuous Delivery	Código sempre pronto para ir para produção. Pode precisar de botão manual.
Continuous Deployment	Cada mudança que passa nos testes vai automaticamente para produção.

Para o projeto da disciplina: vocês vão implementar **Continuous Delivery**.

Pipeline verde = **pronto para deploy**. O botão manual é vocês decidirem quando.

0 pipeline como filtro de qualidade

```
commit → build → test → lint → [aprovação] → deploy
  ↑           |           |           |
  └── feedback a cada etapa ─┘
```

Cada etapa é um **filtro**. Se qualquer uma falha, o processo **para** e a equipe é notificada.

Isso é o **1º Caminho em ação**: fluxo da esquerda para a direita, sem passar defeitos adiante.

"Funciona na minha máquina" → **"Funciona no pipeline"**

DevOps é Lean aplicado à entrega

Lean / Kanban (Sprint 1)	DevOps (Sprint 2)
Fluxo contínuo	Pipeline CI/CD
Reduzir WIP	Branches curtas, merge frequente
Pull system	Deploy on demand
Gestão visual do quadro	Dashboard do pipeline (GitHub Actions)
Muda (desperdício)	Build manual, teste manual, deploy manual
Kaizen (melhoria contínua)	Retrospectivas + blameless post-mortems

Se vocês fizeram a Sprint 1 — quadro Kanban, WIP limits, retrospectiva — vocês já estão praticando a base que DevOps precisa. **Agora vamos automatizar.**

Sprint 2: o que vocês precisam fazer

1. **Pipeline de CI/CD** funcionando no GitHub Actions
2. **Docker** para reprodutibilidade
 - Dockerfile multi-stage, docker-compose
3. **Primeiras métricas DORA** coletadas
 - Frequência de deploy, lead time (pelo menos 2 métricas)
4. **Retrospectiva da Sprint 2** documentada com ações concretas
5. **Vídeo 8min** mostrando pipeline + Docker + métricas + reflexão sobre impacto no fluxo

Prazo: 19/05 (ter) às 23:59

Leituras

- **Aula gravada:** DevOps, CI/CD e Cultura de Entrega — youtu.be/nBd0vsnODp8
- **L8:** Fowler — *Continuous Integration* — martinfowler.com/articles/continuousIntegration.html
- **L9:** Kim et al. — *The DevOps Handbook* (cap. 1-3, no SIGAA)
- **L10:** DORA — *Metrics Guide* — dora.dev/guides/dora-metrics-four-keys

Referências

- Kim, G. et al. (2016) — *The DevOps Handbook* (no SIGAA)
- Humble, J. & Farley, D. (2010) — *Continuous Delivery*
- Forsgren, N. et al. (2018) — *Accelerate*
- DORA Team (2025) — *State of DevOps Report* — dora.dev
- Fowler, M. (2024) — *Continuous Integration* — martinfowler.com
- Kim, G. (2013) — *The Phoenix Project* (romance sobre DevOps)